EEST N.º 1 “Eduardo Ader” — Vicente López
Laboratorio de Programación (LPR)
5.º año — 3.ª división — Grupos A-B

Informe del Proyecto
Calculadora de Edad

Actividad 1: de Revisión Integral (Python & C++)

Integrante(s): [Nombre y Apellido del Estudiante]
Tareas realizadas: [detallar la tarea de cada integrante]

Profesor: Mansilla Muñoz, York Elías
Materia: Laboratorio de Programación
Curso lectivo: 2026
Fecha de elaboración: 12/11/2026

Resumen

Este informe presenta la resolución de la Actividad 1 de revisión integral de la materia Laboratorio de Programación, cuyo objetivo es calcular la edad exacta de una persona a partir de su fecha de nacimiento y de la fecha actual. El programa se implementó en dos entornos (Python y C++) para comparar sus sintaxis y herramientas. En ambos casos se aplicó un filtro de consistencia que rechaza fechas inexistentes en el calendario real (meses fuera de rango, días imposibles y años bisiestos) y fechas de nacimiento posteriores a la fecha actual. Los resultados de las pruebas coinciden en los dos lenguajes.

Palabras clave: cálculo de edad, año bisiesto, filtro de consistencia, Python, C++.

1. Introducción

La propuesta de la actividad consiste en retomar los conceptos de lógica y algoritmos trabajados durante el año anterior y trasladarlos a dos entornos de programación: Python, un lenguaje interpretado de alto nivel, y C++, un lenguaje compilado de bajo nivel. El ejercicio elegido es una calculadora de edad: el usuario ingresa su día, mes y año de nacimiento y el programa devuelve cuántos años tiene.

El problema central no es la resta de años, sino la validación previa de los datos. Un usuario puede escribir el 31 de abril, el 29 de febrero de un año no bisiesto o un año imposible. Por ese motivo el programa incorpora un filtro de consistencia que comprueba, antes de calcular, que la fecha exista realmente en el calendario.

2. Objetivos

2.1. Objetivo general

Desarrollar un programa que calcule la edad exacta de una persona y compare su implementación en Python y en C++.

2.2. Objetivos específicos

- Validar la fecha ingresada verificando rangos de año, mes y día.
- Detectar años bisiestos y ajustar la cantidad de días de febrero.
- Calcular la edad descontando un año cuando el cumpleaños todavía no ocurrió.
- Organizar el repositorio con la estructura de carpetas solicitada.
- Analizar similitudes y diferencias entre ambos lenguajes.

3. Marco teórico

3.1. Año bisiesto

Un año es bisiesto cuando tiene 366 días, es decir, cuando febrero tiene 29 días. La regla del calendario gregoriano se expresa así:

```
(anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)
```

Esto significa que un año divisible por 4 es bisiesto, salvo que también sea divisible por 100; en ese caso solo es bisiesto si además es divisible por 400. Por ejemplo, 2000 fue bisiesto, pero 1900 no lo fue.

3.2. Filtro de consistencia

Un filtro de consistencia es un conjunto de condiciones lógicas que impiden que el usuario ingrese datos imposibles para el sistema. En este proyecto el filtro controla el rango del año (1900 a 2026), el rango del mes (1 a 12), el rango del día (1 a 31), los meses de 30 días y el caso especial de febrero.

3.3. Scaffolding

El scaffolding o esqueleto de código es una plantilla que entrega la estructura básica del programa para que el estudiante se concentre en completar la lógica faltante. En esta actividad el esqueleto incluyó las funciones de validación y el cálculo principal, que fueron completados y ampliados con un control adicional de fechas futuras.

4. Desarrollo

4.1. Estructura del proyecto

El repositorio se organizó con la estructura de carpetas solicitada en la consigna:

```
calculadoraedad/
|-- .gitignore
|-- README.md
|-- LICENSE
|-- docs/
|   `-- InformeProyectoCalculadoraEdad.pdf
|-- proyecto_cpp/
|   `-- main.cpp
`-- proyecto_python/
    `-- main.py
```

4.2. Algoritmo

El algoritmo se divide en tres etapas: (a) lectura de los datos, (b) validación de la fecha y (c) cálculo y salida del resultado. La validación se implementó en funciones separadas para que el código sea reutilizable y legible.

a) Validación de la fecha (Python)

```
def es_bisiesto(anio):
    return (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0)

def es_fecha_valida(d, m, a):
    if a < 1900 or a > 2026:
        return False
    if m < 1 or m > 12:
```

```
        return False
    if d < 1 or d > 31:
        return False
    if m in [4, 6, 9, 11] and d > 30:
        return False
    if m == 2:
        if es_bisiesto(a):
            if d > 29:
                return False
        elif d > 28:
            return False
    return True
```

b) Misma validación (C++)

```
bool esBisiesto(int anio)
{
    return (anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0);
}

bool esFechaValida(int d, int m, int a)
{
    if (a < 1900 || a > 2026) return false;
    if (m < 1 || m > 12) return false;
    if (d < 1 || d > 31) return false;
    if ((m == 4 || m == 6 || m == 9 || m == 11) && d > 30) return false;
    if (m == 2)
    {
        if (esBisiesto(a)) { if (d > 29) return false; }
        else { if (d > 28) return false; }
    }
    return true;
}
```

c) Cálculo de la edad con ajuste

Para obtener la edad exacta no alcanza con restar los años: si la persona todavía no cumplió años en el año actual, hay que descontar un año. La condición compara primero el mes y, si coinciden, compara el día.

```
edad = anioA - anioN

# Ajuste si aún no cumplió años en el año actual
if mesA < mesN or (mesA == mesN and diaA < diaN):
    edad -= 1
```

Además se agregó un control propio: si la fecha de nacimiento es posterior a la fecha actual del ejercicio (08/07/2026), el programa muestra un error en lugar de devolver una edad negativa.

5. Pruebas y resultados

Se ejecutaron los siguientes casos de prueba sobre el programa en Python. Los resultados obtenidos coinciden con los esperados y con la lógica implementada en C++.

Día	Mes	Año	Resultado obtenido	Estado
20	10	2008	Edad calculada: 17 años	Correcto
05	03	2008	Edad calculada: 18 años	Correcto
29	02	2000	Edad calculada: 26 años	Correcto (bisiesto)
29	02	1900	Error: fecha no válida	Correcto
31	04	2000	Error: fecha no válida	Correcto
15	13	2000	Error: fecha no válida	Correcto
01	01	1899	Error: fecha no válida	Correcto
01	12	2026	Error: fecha posterior a la actual	Correcto
abc	01	2000	Error: deben ingresar enteros	Correcto

Tabla 1. Casos de prueba ejecutados y resultados obtenidos.

Las pruebas confirman que el filtro de consistencia funciona: las fechas imposibles se rechazan antes de realizar cualquier cálculo y los años bisiestos se evalúan con la regla correcta (2000 sí, 1900 no). También se verificó que el ajuste de edad descuenta un año cuando el cumpleaños aún no ocurrió en 2026.

6. Comparación entre Python y C++

Aspecto	Python	C++
Ejecución	Interpretado	Compilado
Bloques	Sangría (indentación)	Llaves { }
Tipos de datos	Dinámicos	Estáticos (int, bool)
Lectura de datos	input()	cin >>
Escritura	print() / f-strings	cout <<
Booleano	True / False	true / false
Error de tipo	Excepción ValueError	No aplica en la lectura (cin falla)
Validación	Funciones con and / or	Funciones con && / ||

Tabla 2. Comparación de los dos entornos de programación.

La lógica del algoritmo es idéntica en ambos lenguajes; cambian la sintaxis y las herramientas. Python resulta más breve y permite validar la entrada de datos con excepciones, mientras que C++ exige declarar los tipos y compilar antes de ejecutar, pero ofrece un control más explícito sobre el flujo del programa.

7. Conclusiones

La actividad permitió repasar los conceptos de lógica y algoritmos y aplicar un mismo algoritmo en dos entornos diferentes. La principal dificultad no fue el cálculo de la edad, sino la validación de fechas: es necesario contemplar los meses de distinta duración y la regla completa de los años bisiestos. El ajuste por cumpleaños no cumplido fue el segundo punto importante para obtener la edad exacta.

Como mejora se agregó el control de fechas futuras, que evita resultados negativos. La experiencia muestra que un buen filtro de consistencia hace que el programa sea más seguro y más fácil de probar.

8. Referencias

Python Software Foundation. (2026). *The Python Tutorial*. https://docs.python.org/3/tutorial/

cppreference.com. (2026). *C++ reference*. https://en.cppreference.com/

Mansilla Muñoz, Y. E. (2026). *Actividad 1: de revisión integral (Python & C++) — LPR 5.º 3.º A-B* [Material de clase]. EEST N.º 1 “Eduardo Ader”.

Real Academia Española. (2026). *Diccionario de la lengua española* (23.ª ed.). https://dle.rae.es/